

DSA Master Checklist

The Ultimate 2-Phase Plan for Java Developers — check off each item as you complete it

Phase 1: Data Structures Foundation (The Tools)

Goal: master structural layout, memory allocation, and basic CRUD manipulation. Don't jump into hard LeetCode problems yet.

1. Linear Data Structures

- ☐ Arrays & Strings — index-based access, sequential iteration, `.length()`, `.charAt()`, `.substring()`
- ☐ Stacks & Queues — LIFO / FIFO principles; use `ArrayDeque` (avoid the legacy `Stack` class)
- ☐ Linked Lists (Singly & Doubly) — write insert / delete / traverse from scratch without losing node references

2. Non-Linear Data Structures

- ☐ Hash Maps & Sets — hashing mechanics; store/retrieve key-value pairs in $O(1)$
- ☐ Trees (Binary Search Trees) — root/leaf/child nodes, basic insertion and lookup
- ☐ Graphs — nodes & edges; represent via Adjacency List (`ArrayList<ArrayList<Integer>>` or `HashMap<Integer, List<Integer>>`)
- ☐ Tries — prefix tree structure for fast string prefix lookups (autocomplete, dictionary search)

3. Java Collections Framework (JCF)

- ☐ `ArrayList` — dynamic sorting
- ☐ `ArrayDeque` — Stacks & Queues
- ☐ `HashMap` / `HashSet` — constant-time lookups
- ☐ `PriorityQueue` — with custom lambda comparators, e.g. `(a, b) -> b - a`

Phase 2: Algorithmic Problem-Solving (The Patterns)

Goal: transition from data containers to logical patterns. Pick one pattern, learn its template, and stick with it for 5-7 consecutive days before switching (80/20 rule).

☐ 1. Two Pointers (Target: $O(n)$)

Iterating from both ends or distinct positions of a sorted array/list to find a pair or rearrange elements.

- ☐ Two Sum II

- ☐ 3Sum
- ☐ Sort Colors

☐ **2. Fast & Slow Pointers** (Target: $O(n)$)

Two pointers moving at different speeds to detect loops, structural midpoints, or cycles.

- ☐ Linked List Cycle
- ☐ Find the Duplicate Number
- ☐ Palindrome Linked List

☐ **3. Sliding Window** (Target: $O(n)$)

Expanding/shrinking a contiguous sub-segment (subarray/substring) to evaluate window criteria.

- ☐ Max Sum Subarray of Size K
- ☐ Longest Substring Without Repeating Chars
- ☐ Minimum Window Substring

☐ **4. Kadane's Pattern** (Target: $O(n)$)

Single-pass iterative tracking to compute maximum or minimum contiguous global sums.

- ☐ Maximum Subarray
- ☐ Maximum Product Subarray
- ☐ Maximum Absolute Sum

☐ **5. Prefix Sum** (Target: $O(n)$ time, $O(n)$ space)

Pre-calculating cumulative arrays to answer coordinate interval or subarray sum queries instantly.

- ☐ Subarray Sum Equals K
- ☐ Find Pivot Index
- ☐ Contiguous Array

☐ **6. Merge Intervals** (Target: $O(n \log n)$ due to sorting)

Sorting and resolving overlapping ranges, schedules, or coordinate time-slots.

- ☐ Merge Intervals
- ☐ Insert Interval
- ☐ Minimum Meeting Rooms

☐ **7. In-Place Reversal (Linked List)** (Target: $O(n)$ time, $O(1)$ space)

Re-linking node connections sequentially without allocating auxiliary memory objects.

- ☐ Reverse Linked List

- ☐ Reverse Linked List II
- ☐ Reverse Nodes in k-Group

☐ **8. Monotonic Stack** (Target: $O(n)$)

Maintaining a strictly increasing/decreasing stack structure to find nearest immediate maxima/minima.

- ☐ Next Greater Element
- ☐ Daily Temperatures
- ☐ Remove K Digits

☐ **9. Hash Map Tracking** (Target: $O(n)$ time, $O(n)$ space)

Using key-value maps as an instantaneous digital look-back memory bank during a single linear pass.

- ☐ First Unique Character
- ☐ Ransom Note
- ☐ Subarray Sum Equals K

☐ **10. Binary Search (Modified)** (Target: $O(\log n)$)

Dividing search horizons logarithmically; applied to rotated arrays and "Search Space of Answers."

- ☐ Search in Rotated Sorted Array
- ☐ Koko Eating Bananas
- ☐ Book Allocation Problem

☐ **11. Heaps / Top 'K'** (Target: $O(n \log k)$)

Utilizing Min-Heaps/Max-Heaps to dynamically track high-frequency or minimum priority elements.

- ☐ Kth Largest Element
- ☐ Top K Frequent Elements
- ☐ Find Median from Data Stream

☐ **12. Recursion & Backtracking** (Target: Exponential — $O(2^n)$ / $O(n!)$)

Systematically generating and exploring all branch pathways, with an immediate "undo" state reversal step.

- ☐ Permutations
- ☐ Combination Sum
- ☐ Generate Parentheses

☐ **13. Tree Traversals** (Target: $O(n)$)

Utilizing specialized depth strategies (DFS) or level-by-level queuing (BFS) on structural layouts.

- ☐ Binary Tree Level Order Traversal

- ☐ Lowest Common Ancestor
- ☐ Validate BST

☐ **14. Graph Routings** (Target: $O(V+E)$)

Traversing interconnected nodes via structural connectivity matrices or tracking optimal weights.

- ☐ Number of Islands
- ☐ Rotten Oranges
- ☐ Dijkstra's Algorithm

☐ **15. Dynamic Programming** (Target: *Dependent on states*)

Optimizing recursive workflows by breaking them down and caching sub-problem results.

- ☐ Climbing Stairs
- ☐ House Robber
- ☐ 0/1 Knapsack

☐ **16. Bit Manipulation** (Target: $O(1)$)

Evaluating sequences down to the bit layer; modifying bits directly (AND, OR, XOR, shifts) to solve frequency problems and flags.

- ☐ Single Number
- ☐ Number of 1 Bits
- ☐ Counting Bits

Execution Strategy: "One Question Daily"

The 80/20 practice rule: don't jump across patterns randomly. Choose one pattern, learn its structural template, and stick with it for 5-7 consecutive days before changing topics.

Daily Problem Lifecycle

- ☐ 1. Read & Deconstruct (10 min) — identify keywords ("contiguous subarray" → Sliding Window/Kadane's; "shortest path" → Graph BFS)
- ☐ 2. Brute Force Scripting (15 min) — write/verbalize the brute force solution ($O(n^2)$ or nested loops) before optimizing
- ☐ 3. Pattern Optimization (25 min) — apply pointers, heaps, or maps to eliminate duplicate work and lower complexity
- ☐ 4. Java Code Review (10 min) — verify memory footprint; use StringBuilder for string edits, PriorityQueue with comparators for custom objects